Avoiding Schema Cache Errors

When Active Record starts up, for each model that's backed by a database table, the table fields are scanned and cached as the *Schema Cache*[10] for as long as the application is running.

Dropping a database column without considering Active Record and active instances of the application means that a schema change may cause an application error where there is code that tries to write or read from a column that no longer exists.

This isn't required for every change; refer to the documentation for specific examples. This section is intended to familiarize you with the Schema Cache concept and how it works so that you can determine whether this risk exists for your change. This type of error would be more common at a higher scale as well, where there may be multiple running copies of the application.

To drop a column safely, first work in Active Record. Use the Active Record `ignored_columns`[11] mechanism to add the field you're planning to drop first. Add the field there, commit and release your change, making sure the application restarts.

A field added to `ignored_columns` can safely be dropped from the database without concern that it's still in the Schema Cache.

Backfilling Large Tables Without Downtime

Over time your application structure will evolve. You may introduce a new column that will be NULL by default, and then populated with a value for the purposes of the application.

Populating values like this is usually called *backfilling*. As your table row counts grow large, backfills take longer and can become more complicated and risky from an operational perspective.

When possible, backfills could be made optional if the application can fall back to a default value or some other value when the column is NULL. When that's not possible, you'll need to backfill.

While you may conduct backfills from a Ruby application using a Rake task, in this section you'll conduct backfills using SQL, connected to your database using psql.

10. https://api.rubyonrails.org/classes/ActiveRecord/ConnectionAdapters/SchemaCache.html
11. https://www.bigbinary.com/blog/rails-5-adds-active-record-ignored-columns